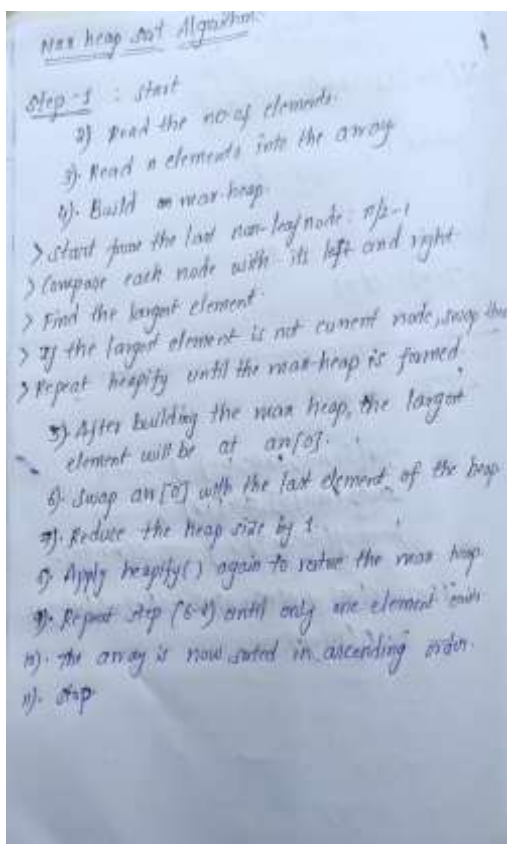


PRACTICAL - 3

AIM :Implementation of max-heap sort algorithm

Max heap sort Algorithm :



CODE:

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;    int left = 2 * i + 1;
    int right = 2 * i + 2;    if (left < n &&
    arr[left] > arr[largest])
```

```
largest = left;

if (right < n && arr[right] > arr[largest])
    largest = right;

if (largest != i)
{
    swap(arr[i], arr[largest]);
    heapify(arr, n, largest);
}
}

void heapSort(int arr[], int n) {
    // Build Max Heap    for (int i
= n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Heap Sort    for (int i =
n - 1; i > 0; i--)
    {
        swap(arr[0],
arr[i]);    heapify(arr,
i, 0);
    }
}

int main() {
    int n;    int
arr[100];

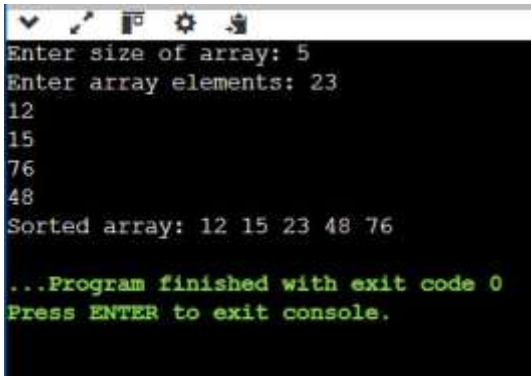
    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    heapSort(arr, n);
```

```
cout << "Sorted array: ";  
for (int i = 0; i < n; i++)  
    cout << arr[i] << " ";  
  
return 0;  
}
```

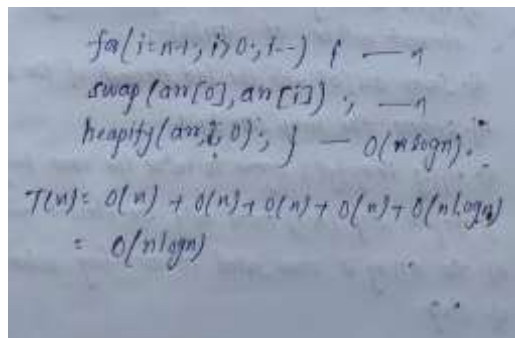
OUTPUT:



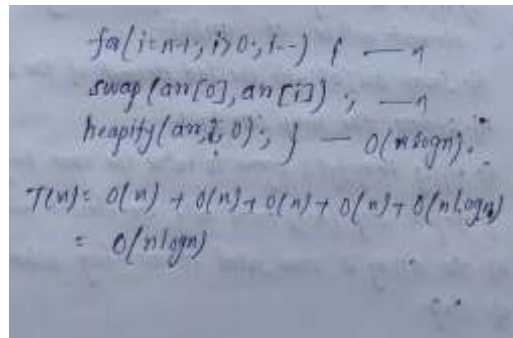
```
Enter size of array: 5  
Enter array elements: 23  
12  
15  
76  
48  
Sorted array: 12 15 23 48 76  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

TIME COMPLEXITY:

1)best case :

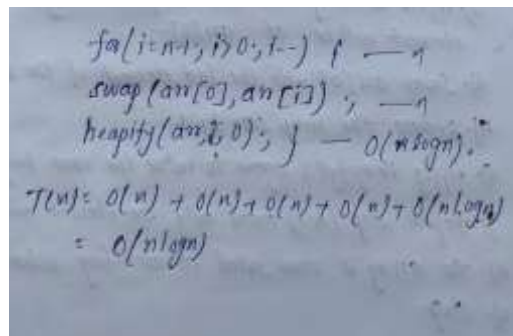

$$\begin{aligned} & \text{for } (i = n-1; i > 0; i--) \{ \text{---} \} \\ & \text{swap}(arr[0], arr[i]); \text{---} \\ & \text{heapify}(arr, i, 0); \} \text{---} O(n \log n) \\ T(n) &= O(n) + O(n) + O(n) + O(n) + O(n \log n) \\ &= O(n \log n) \end{aligned}$$

2)average case :



for $i = n-1, i > 0, i--$ {
 $\text{swap}(a[n], a[i])$;
 $\text{heapify}(a, i, 0)$; }
 $T(n) = O(n) + O(n) + O(n) + O(n) + O(n \log n)$
 $= O(n \log n)$

3)worst case :



for $i = n-1, i > 0, i--$ {
 $\text{swap}(a[n], a[i])$;
 $\text{heapify}(a, i, 0)$; }
 $T(n) = O(n) + O(n) + O(n) + O(n) + O(n \log n)$
 $= O(n \log n)$